ProbOS — Month 3, Week 13
PDSL v0.2: Control Flow

Nisong Monyimba

July 8, 2026

Week 13 goal

Extend PDSL from v0.1 to v0.2 by adding comparison operators and conditional expressions – the first control-flow constructs PDSL has ever supported.

Scope confirmed via direct inspection of `grammar.lark` before writing this plan: PDSL v0.1 has ZERO comparison operators and ZERO conditional constructs – only arithmetic (+, -, *, /, **, unary +/-) and function calls.

Design decision, stated up front: this adds EXPRESSION-level conditionals (`if cond then expr else expr`), NOT statement-level control flow or loops. PDSL describes a model's per-step drift declaratively; the outer simulation engine already handles the step-by-step loop. An expression-level conditional (for clamping, thresholding, or piecewise physical behavior) is the right granularity.

Day-by-day plan

Day 1 – Grammar design

- Add comparison operators to `grammar.lark`: `<`, `>`, `<=`, `>=`, `==`, `!=` – at a new precedence level between `expr` and a new top-level `condition` rule
- Add a conditional expression rule: `"if" condition "then" expr "else" expr -> conditional`
- Test the grammar in isolation (Lark parse-tree inspection) before touching `parser.py` or `codegen.py`

Day 2 – AST nodes and parser

- Add `Comparison` and `Conditional` AST node types to `ast_nodes.py`, matching existing node style
- Extend `parser.py` to build these new AST nodes

Day 3 – Code generation

- Extend `codegen.py` to emit correct, VECTORISED Python: a naive `if/else` would not vectorise correctly over N particles – must generate `xp.where(cond, expr_true, expr_false)`, matching the vectorisation discipline already used throughout the hand-written kernel models

Day 4 – Test coverage

- Grammar-level, codegen-level, and end-to-end tests: a genuinely new PDSL model using a conditional compiles and runs correctly through the full pipeline, matching a hand-written Python equivalent

Day 5 – Existing model regression check

- Confirm ALL existing PDSL v0.1 models/examples still parse and compile correctly – strictly additive, not breaking

Day 6 – pre_week_audit.sh + CI

- Full audit pass

Day 7 – Documentation + retrospective

- Update `docs/vision/main.tex` if it references PDSL version status
- Retrospective appended once complete

Exit criteria

PDSL v0.2 correctly parses, compiles, and executes conditional expressions with comparison operators, generating correctly- vectorised NumPy/CuPy code (`xp.where`), with zero regressions to any existing v0.1 model.

What was actually accomplished (retrospective)

- PDSL v0.2 shipped: comparison operators (¡, ¿, ¡=, ¿=, ==, !=) and expression-level conditionals (if cond then expr else expr) – the first control-flow constructs PDSL has ever supported.
- An honest correction was made to this plan document's own stated intention during Day 3: the plan said codegen "must generate xp.where(), matching CuPy dispatch" – but direct inspection of the actual codegen.py showed PDSL v0.1 has no xp/CuPy dispatch at all, only plain `np.`. np.where() was the correct, properly-scoped choice.
- Genuine end-to-end proof of correct vectorisation: 5 particles above a threshold and 5 below, in the SAME `forward_batch()` call, correctly diverged based on their own individual state values. A boundary test confirmed strict comparison semantics.
- Two real, unrelated bugs were found and fixed, surfaced by `check_ci.sh`'s corrected, accurate failure reporting:
  1. `check_ci.sh` itself had a genuine race condition, permanently fixed by matching the exact commit SHA with a polling retry loop.
  2. CI's C++ dependency installation failed due to a broken Microsoft azure-cli apt repository – a genuine, external infrastructure issue. Fixed robustly (grep by repo file CONTENT, not a guessed filename).

What went right

1. Verifying assumptions against the actual current codebase caught the xp/CuPy scope-creep risk before any code was written.
2. A test failure was correctly diagnosed as being in the TEST SCRIPT's PDSL syntax, not the actual codegen fix.
3. check_ci.sh's fix immediately proved its own value: it correctly caught a genuine CI failure, leading directly to finding and fixing the Microsoft-repo infrastructure issue.

Numbers at Week 13 close

- Python tests: 408/408 passing
- mypy strict: 0 errors
- ruff: 0 warnings
- Zero regressions to any existing PDSL v0.1 model